

Algoritmo AGE aplicado al QAP

Explicación extendida, fundamentos teóricos y procedimiento con *random keys*

Resumen

Este documento describe de manera detallada el funcionamiento del algoritmo AGE (un esquema evolutivo estacionario) y su aplicación al Quadratic Assignment Problem (QAP) mediante codificación *random keys*. Además de la adaptación práctica, se incluyen fundamentos teóricos necesarios para comprender el paradigma de Algoritmos Genéticos, la diferencia entre modelos generacionales y estacionarios, y el rol de cada operador evolutivo (selección, cruce y mutación) en términos de exploración/explotación. Se aportan pseudocódigo y diagramas orientativos en TikZ para entender el flujo completo.

Índice

1. Marco teórico: computación evolutiva y Algoritmos Genéticos	2
1.1. Computación evolutiva: idea general	2
1.2. Algoritmos Genéticos (GA): definición y componentes	2
1.3. Presión selectiva, exploración y explotación	2
1.4. Reemplazo generacional vs estacionario	2
1.5. Qué significa que AGE sea estacionario	3
1.6. Selección por torneo (por qué es tan común)	3
1.7. Cruce y mutación: roles complementarios	3
1.8. Comentario teórico útil para defensa: por qué funcionan (a alto nivel)	3
2. Contexto: QAP y <i>random keys</i>	4
2.1. QAP (recordatorio breve)	4
2.2. Por qué usar <i>random keys</i> para AGE	4
2.3. Diagrama: <i>random keys</i> (TikZ)	4
3. Fundamentos del algoritmo AGE	4
3.1. Qué es AGE en términos operativos	4
3.2. Componentes esenciales	4
3.3. Intuición dinámica: exploración vs explotación	5
4. AGE aplicado a QAP: procedimiento completo	6
4.1. Datos y estructuras	6
4.2. Evaluación eficiente del QAP	6
4.3. Pseudocódigo detallado (AGE + QAP + <i>random keys</i>)	6
4.4. Qué significa cada paso al defender	7
5. Diagrama de flujo (TikZ): AGE aplicado a QAP	7
6. Preguntas típicas de defensa (y cómo responder)	7
7. Conclusión	8

1. Marco teórico: computación evolutiva y Algoritmos Genéticos

1.1. Computación evolutiva: idea general

La computación evolutiva agrupa una familia de métodos de optimización inspirados en procesos evolutivos (selección natural, recombinación y mutación). Comparten varios rasgos:

- Mantienen una **población** de soluciones candidatas.
- Iteran aplicando operadores que introducen variación (mutación/recombinación).
- Aplican un mecanismo de **selección** guiado por el valor de la función objetivo (fitness).

A diferencia de métodos basados en gradiente, no requieren derivadas y toleran mejor funciones no convexas, no suaves o altamente multimodales.

1.2. Algoritmos Genéticos (GA): definición y componentes

Un Algoritmo Genético (GA) es un algoritmo evolutivo en el que:

- El **genotipo** es la representación interna (cromosoma).
- El **fenotipo** es la solución interpretada en el problema.
- El **fitness** cuantifica la calidad (minimización o maximización).

Un GA típico se describe mediante las piezas siguientes:

1. **Representación:** cómo codificamos una solución (binaria, real, permutación, etc.).
2. **Inicialización:** creación de una población inicial (aleatoria o heurística).
3. **Evaluación:** cálculo de fitness para cada individuo.
4. **Selección:** elige padres con sesgo hacia mejores fitness.
5. **Cruce (crossover):** combina dos padres para producir descendencia.
6. **Mutación:** altera aleatoriamente genes para introducir diversidad.
7. **Reemplazo:** decide qué individuos sobreviven a la siguiente población.
8. **Criterio de parada:** número de evaluaciones, tiempo, estancamiento, etc.

1.3. Presión selectiva, exploración y explotación

Un GA debe equilibrar:

- **Explotación:** concentrarse en regiones prometedoras (selección fuerte, reemplazo elitista).
- **Exploración:** mantener diversidad para evitar óptimos locales (mutación, recombinación, selección menos agresiva).

Si la presión selectiva es excesiva, puede ocurrir **convergencia prematura**; si es demasiado baja, el progreso puede ser lento.

1.4. Reemplazo generacional vs estacionario

Existen dos grandes modelos de reemplazo:

GA generacional. En cada generación se crea una población completa de descendientes y se reemplaza (total o parcialmente) la población anterior. Ventaja: estructura simple. Riesgo: puede perder diversidad o buenos individuos si no hay elitismo.

GA estacionario (steady-state). Se generan pocos descendientes por iteración (1–2 típicamente) y se reemplaza a pocos individuos. Ventaja: cambios más graduales, posibilidad de mantener diversidad y estabilidad, y a menudo mejor explotación incremental.

1.5. Qué significa que AGE sea estacionario

En tu contexto, AGE se entiende como un algoritmo genético con reemplazo estacionario:

- No hay “generaciones” completas en el sentido de reemplazo total.
- Cada iteración produce una o dos soluciones nuevas.
- El reemplazo suele actuar sobre el peor, o sobre individuos específicos (por ejemplo, los más similares o los menos aptos).

Esto es clave en defensa: cuando te pregunten “¿qué cambia respecto a un GA clásico?”, la respuesta es el esquema de reemplazo y el ritmo de actualización de la población.

1.6. Selección por torneo (por qué es tan común)

La selección por torneo elige k individuos al azar y selecciona el mejor (mínimo coste).
Propiedades:

- Es fácil de implementar y paralelizar.
- Controla la presión selectiva con k :
 - k pequeño: más exploración.
 - k grande: más explotación.
- No requiere normalizar fitness ni suposiciones de escala.

1.7. Cruce y mutación: roles complementarios

Cruce. Explora “combinando” material genético de buenos individuos. En representaciones continuas, mezcla componentes reales para crear hijos intermedios o extrapolados.

Mutación. Garantiza diversidad y capacidad de escapar de óptimos locales. En representaciones continuas, típicamente se implementa como perturbación (ruido) controlada por un parámetro σ o tasas por componente.

1.8. Comentario teórico útil para defensa: por qué funcionan (a alto nivel)

Sin entrar en formalismos duros, una explicación defendible es:

- La selección incrementa la probabilidad de reproducir bloques de construcción “buenos”.
- El cruce recombina esos bloques en nuevas configuraciones.
- La mutación inyecta variación que evita que el proceso se “congele”.
- El reemplazo asegura que la población retenga (en promedio) soluciones de mejor calidad.

2. Contexto: QAP y *random keys*

2.1. QAP (recordatorio breve)

Dadas matrices de flujos $F \in \mathbb{R}^{n \times n}$ y distancias $D \in \mathbb{R}^{n \times n}$, una solución es una permutación $p \in \mathcal{S}_n$ que asigna cada instalación i a una localización $p(i)$. El coste se define como:

$$C(p) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} F_{ij} D_{p(i)p(j)} \quad (\text{minimización}).$$

2.2. Por qué usar *random keys* para AGE

AGE suele definirse sobre cromosomas en un espacio donde sus operadores (cruce/mutación) tienen sentido. Para reutilizar un AGE continuo sin rediseñar operadores de permutación, se usa:

$$x \in \mathbb{R}^n \Rightarrow p = \text{argsort}(x).$$

La permutación se obtiene ordenando índices por el valor de x . Así:

- AGE opera sobre x (espacio continuo).
- El QAP se evalúa sobre p (espacio discreto de permutaciones).

2.3. Diagrama: *random keys* (TikZ)

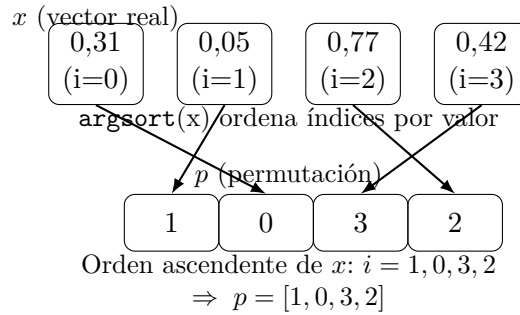


Figura 1: Decodificación *random keys*: el orden de los valores de x induce una permutación válida.

3. Fundamentos del algoritmo AGE

3.1. Qué es AGE en términos operativos

En este contexto, **AGE es un algoritmo evolutivo** con las características siguientes:

- Mantiene una **población** P de tamaño μ .
- En cada iteración genera un número pequeño de descendientes (típicamente 1 o 2).
- El **reemplazo es estacionario**: se sustituyen pocos individuos en cada paso, lo que introduce una presión selectiva controlada.

3.2. Componentes esenciales

Para entender AGE al 100 %, conviene aislar sus “piezas”:

(1) **Representación** Un individuo es el cromosoma $x \in \mathbb{R}^n$.

(2) **Inicialización** Se generan μ vectores reales, por ejemplo uniformes en $[0, 1]^n$.

(3) **Evaluación** Para cada x :

$$p = \text{argsort}(x), \quad \text{fitness}(x) = C(p).$$

La evaluación es determinista si F y D lo son.

(4) **Selección** Se escogen padres para reproducirse. Una opción común es **torneo**:

- se eligen k individuos al azar,
- gana el de menor coste (minimización).

A mayor k , mayor presión selectiva.

(5) **Cruce (recombinación)** En AGE continuo, el cruce mezcla valores reales de los padres. Ejemplos típicos:

- cruce aritmético: $h = \alpha p_1 + (1 - \alpha)p_2$,
- BLX- α o similares.

(La elección concreta depende de tu implementación, pero el concepto es el mismo: *explorar combinando material genético*.)

(6) **Mutación** Introduce variación:

- **añadir ruido pequeño**: $x_i \leftarrow x_i + \sigma \varepsilon$,
- mutación por componente con probabilidad p_m .

Importante: en *random keys*, cambios en x cambian el orden relativo y por tanto la permutación.

(7) **Reemplazo (estacionario)** AGE suele reemplazar:

- al **peor individuo**, o
- mediante “mejor hijo compite contra peor / o contra uno similar”.

La regla más sencilla (y defendible) es: **si el hijo mejora al peor, lo reemplaza**.

3.3. Intuición dinámica: exploración vs explotación

- **Explotación**: selección favorece individuos buenos; el cruce tiende a combinar regiones prometedoras.
- **Exploración**: mutación evita estancamiento y permite saltos.
- **Estacionariedad**: al reemplazar pocos individuos por iteración, se conserva diversidad más tiempo que en reemplazos generacionales agresivos.

4. AGE aplicado a QAP: procedimiento completo

4.1. Datos y estructuras

- Población $P = \{x^{(1)}, \dots, x^{(\mu)}\}$, $x^{(i)} \in [0, 1]^n$.
- Fitness asociado $f^{(i)} = \text{fitness}(x^{(i)})$.
- Presupuesto: número máximo de evaluaciones o tiempo máximo.

4.2. Evaluación eficiente del QAP

Aunque el pseudocódigo clásico usa doble bucle, en práctica se puede:

1. calcular $p = \text{argsort}(x)$,
2. formar $D_p = D[p, :][:, p]$,
3. devolver $\sum_{i,j} F_{ij}(D_p)_{ij}$.

Esto mantiene el significado matemático y reduce el coste en implementaciones vectorizadas.

4.3. Pseudocódigo detallado (AGE + QAP + random keys)

```
procedure AGE_QAP(F, D, mu, k_torneo, max_evals)
  # 1) Inicialización
  P <- {mu vectores x ~ Uniforme([0,1]^n)}
  evaluar cada x: p = argsort(x), fitness = C(p)
  evals <- mu

  while evals < max_evals:
    # 2) Selección (torneo)
    padre1 <- torneo(P, k_torneo)
    padre2 <- torneo(P, k_torneo)

    # 3) Reproducción
    (h1, h2) <- cruce(padre1, padre2)
    h1 <- mutacion(h1)
    h2 <- mutacion(h2)

    # 4) Evaluación
    f1 <- C(argsort(h1))
    f2 <- C(argsort(h2))
    evals <- evals + 2

    # 5) Reemplazo estacionario (ejemplo: reemplazar peores si mejoran)
    insertar (h1,f1) si mejora a algún individuo (p.ej. al peor)
    insertar (h2,f2) si mejora a algún individuo (p.ej. al peor)

  return mejor individuo de P
end procedure
```

4.4. Qué significa cada paso al defender

Cuando te pregunten “¿por qué funciona?”, la respuesta razonada suele ser:

- **Inicialización** cubre el espacio; en random keys genera rankings variados.
- **Selección** aumenta probabilidad de reproducir soluciones buenas.
- **Cruce** explora recombining rankings inducidos por los padres.
- **Mutación** introduce nuevos órdenes relativos y evita estancamiento.
- **Reemplazo estacionario** hace progreso monotónico (si sólo aceptas mejoras) y estabiliza diversidad.

5. Diagrama de flujo (TikZ): AGE aplicado a QAP

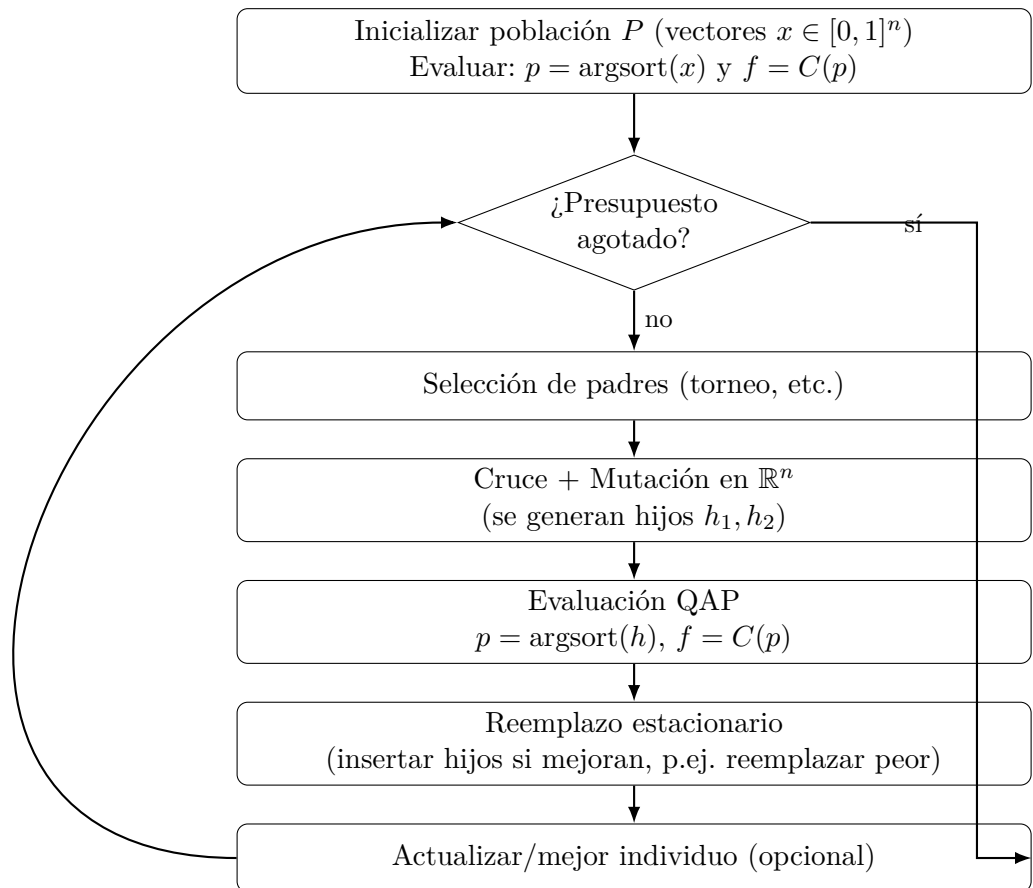


Figura 2: Flujo conceptual del AGE aplicado al QAP con codificación *random keys*.

6. Preguntas típicas de defensa (y cómo responder)

(1) ¿Qué optimiza AGE realmente: x o p ?

AGE optimiza x en el espacio continuo, pero el objetivo real se evalúa sobre $p = \text{argsort}(x)$. Por tanto, optimiza la función compuesta:

$$f(x) = C(\text{argsort}(x)).$$

(2) ¿Por qué `argsort` produce una permutación válida?

Porque ordena índices $0..n - 1$ sin repetición; el resultado siempre es una biyección.

(3) ¿Qué efecto tiene una mutación pequeña en x ?

Puede:

- no cambiar el orden relativo (misma permutación),
- o invertir el orden de algunos componentes cercanos (cambio local en la permutación).

Eso introduce una noción de “vecindad” en el espacio continuo.

(4) ¿Qué controla la presión selectiva?

El tamaño de torneo k y la política de reemplazo. Un k alto y reemplazar siempre al peor aumenta explotación pero puede reducir diversidad.

7. Conclusión

AGE aplicado al QAP con *random keys* permite reutilizar operadores continuos, manteniendo una evaluación combinatoria determinista. Entender cada fase (selección, variación, evaluación y reemplazo estacionario) permite argumentar su comportamiento y justificar decisiones de parámetros y diseño durante una defensa.